

Fishing competition system

Dokumentacja techniczna

1. Cel projektu

Celem projektu było stworzenie systemu wspomagającego organizację zawodów wędkarskich. System umożliwia zarządzanie zawodami, zawodnikami, drużynami oraz wynikami zawodów.

2. Wykorzystane technologie

Backend	Node.js
	Express
Baza danych	PostgreSQL
ORM	Prisma ORM
Autoryzacja	JWT (JSON Web Token)
Dokumentacja	Swagger OpenAPI
Konteneryzacja	Docker

3. Architektura systemu

Aplikacja została zbudowana w modelu klient–serwer. Część serwerowa wykorzystuje architekturę warstwową składającą się z warstwy DTO, Controller, Service oraz dostępu do danych realizowanego przez Prisma ORM i bazę PostgreSQL.

Poszczególne warstwy systemu:

- DTO – walidacja danych wejściowych
- Controller – obsługa żądań HTTP
- Service – logika biznesowa
- Prisma ORM – komunikacja z bazą danych
- PostgreSQL – przechowywanie danych

Takie podejście pozwala oddzielić logikę biznesową od warstwy prezentacji i dostępu do danych.

4. Model danych

System składa się z następujących encji:

- User
- Competition
- Competitor
- Team
- TeamMember
- Round
- Sector
- Start
- Result

Relacje:

Competition – Team: relacja 1:N

Jedne zawody mogą posiadać wiele drużyn, a każda drużyna należy do jednych zawodów.

Competition – Round: relacja 1:N

Jedne zawody mogą posiadać wiele tur, a każda tura należy do jednych zawodów.

Team – TeamMember: relacja 1:N

Jedna drużyna może posiadać wielu członków.

Competitor – TeamMember: relacja 1:N

Jeden zawodnik może występować w wielu wpisach składu drużyny.

Round – Sector: relacja 1:N

Jedna tura może posiadać wiele sektorów.

Round – Start: relacja 1:N

Jedna tura może posiadać wiele startów zawodników.

Sector – Start: relacja 1:N

Jeden sektor może zawierać wiele startów.

Competitor – Start: relacja 1:N

Jeden zawodnik może mieć wiele startów.

Start – Result: relacja 1:1

Jeden start może mieć jeden przypisany wynik.

5. Diagram ERD



6. Autoryzacja i role

System wykorzystuje JWT Authentication.

Dostępne role:

- ADMIN
- ORGANIZER
- JUDGE
- USER

Role ograniczają dostęp do wybranych endpointów systemu.

7. Dokumentacja API

Dokumentacja endpointów została przygotowana w standardzie OpenAPI 3.0 z wykorzystaniem Swagger UI.

Dokumentacja dostępna jest pod adresem:

<http://localhost:3000/api-docs>

8. Opis endpointów

Moduł	Metoda	Endpoint	Dostęp	Opis
Auth	POST	/api/auth/register	Publiczny	Rejestracja użytkownika
Auth	POST	/api/auth/login	Publiczny	Logowanie użytkownika i zwrócenie tokenu JWT
Auth	GET	/api/auth/me	Zalogowany użytkownik	Pobranie danych aktualnie zalogowanego usera
Competitions	GET	/api/competitions	Publiczny	Pobranie listy zawodów
Competitions	GET	/api/competitions/{id}	Publiczny	Pobranie zawodów po ID
Competitions	POST	/api/competitions	ADMIN, ORGANIZER	Utworzenie zawodów
Competitions	PUT	/api/competitions/{id}	ADMIN, ORGANIZER	Aktualizacja zawodów
Competitions	DELETE	/api/competitions/{id}	ADMIN	Usunięcie zawodów
Competitors	GET	/api/competitors	Publiczny	Pobranie listy zawodników
Competitors	GET	/api/competitors/{id}	Publiczny	Pobranie zawodnika po ID
Competitors	POST	/api/competitors	ADMIN, ORGANIZER	Utworzenie zawodnika
Competitors	PUT	/api/competitors/{id}	ADMIN, ORGANIZER	Aktualizacja zawodnika
Competitors	DELETE	/api/competitors/{id}	ADMIN	Usunięcie zawodnika
Teams	GET	/api/teams	Publiczny	Pobranie listy drużyn
Teams	GET	/api/teams/{id}	Publiczny	Pobranie drużyny po ID
Teams	POST	/api/teams	ADMIN, ORGANIZER	Utworzenie drużyny
Teams	PUT	/api/teams/{id}	ADMIN, ORGANIZER	Aktualizacja drużyny
Teams	DELETE	/api/teams/{id}	ADMIN	Usunięcie drużyny
Team Members	GET	/api/teams/{teamId}/members	Publiczny	Pobranie składu drużyny
Team Members	POST	/api/teams/{teamId}/members	ADMIN, ORGANIZER	Dodanie zawodnika do drużyny
Team Members	DELETE	/api/teams/{teamId}/members/{competitorId}	ADMIN, ORGANIZER	Usunięcie zawodnika z drużyny
Rounds	GET	/api/rounds	Publiczny	Pobranie listy tur
Rounds	GET	/api/rounds/{id}	Publiczny	Pobranie tury po ID
Rounds	POST	/api/rounds	ADMIN, ORGANIZER	Utworzenie tury

Rounds	PUT	/api/rounds/{id}	ADMIN, ORGANIZER	Aktualizacja tury
Rounds	DELETE	/api/rounds/{id}	ADMIN, ORGANIZER	Usunięcie tury
Sectors	GET	/api/sectors	Publiczny	Pobranie listy sektorów
Sectors	GET	/api/sectors/{id}	Publiczny	Pobranie sektora po ID
Sectors	POST	/api/sectors	ADMIN, ORGANIZER	Utworzenie sektora
Sectors	PUT	/api/sectors/{id}	ADMIN, ORGANIZER	Aktualizacja sektora
Sectors	DELETE	/api/sectors/{id}	ADMIN, ORGANIZER	Usunięcie sektora
Starts	GET	/api/starts	Publiczny	Pobranie listy startów
Starts	GET	/api/starts/{id}	Publiczny	Pobranie startu po ID
Starts	POST	/api/starts	ADMIN, ORGANIZER	Utworzenie startu zawodnika
Starts	PUT	/api/starts/{id}	ADMIN, ORGANIZER, JUDGE	Aktualizacja startu
Starts	DELETE	/api/starts/{id}	ADMIN, ORGANIZER	Usunięcie startu
Results	GET	/api/results	Publiczny	Pobranie listy wyników
Results	GET	/api/results/{id}	Publiczny	Pobranie wyniku po ID
Results	GET	/api/results/round/{id}	Publiczny	Pobranie wyników dla wybranej tury
Results	GET	/api/results/calculate/round/{roundId}/sector/{sectorId}	Publiczny	Obliczenie wyników sektora bez zapisywania
Results	POST	/api/results	ADMIN, ORGANIZER, JUDGE	Utworzenie wyniku
Results	PUT	/api/results/{id}	ADMIN, ORGANIZER, JUDGE	Aktualizacja wyniku
Results	DELETE	/api/results/{id}	ADMIN, ORGANIZER	Usunięcie wyniku
Results	POST	/api/results/generate/round/{roundId}/sector/{sectorId}	ADMIN, ORGANIZER, JUDGE	Wygenerowanie i zapisanie wyników sektora

9. Instrukcja uruchomienia systemu

Wymagania

- Docker Desktop
- Git

Pobranie projektu

```
git clone <adres_repozytorium>  
cd fishing-competition-system
```

Uruchomienie systemu

Uruchomienie aplikacji wraz z bazą danych:

```
docker compose up --build
```

Migracje bazy danych

W osobnym terminalu:

```
docker exec -it fishing-api npx prisma migrate deploy
```

Dostęp do aplikacji

API:

<http://localhost:3000/api>

Swagger:

<http://localhost:3000/api-docs>

Zatrzymanie systemu

```
docker compose down
```

Dane konfiguracyjne

Aplikacja wykorzystuje zmienne środowiskowe:

- PORT
- DATABASE_URL
- JWT_SECRET
- JWT_EXPIRES_IN

Przykładowa konfiguracja znajduje się w pliku `.env.example`.

